AWS API Gateway Lab Part 3: Create HTTP API and Route to Lambda

To Begin with the Lab

Summary of the Lab

In this part of the lab, an **HTTP API** was created in **API Gateway** to expose the backend Lambda functions as real API routes. The lab connected the frontend request flow to the three Lambda functions prepared earlier: create user, get user, and delete user. A new API named **UserAPI** was created, and routes were mapped so that POST /user invoked **createUserFunction**, GET /user/{userId} invoked **getUserFunction**, and DELETE /user/{userId} invoked **deleteUserFunction**. This layer acted as the bridge between the client application and the Lambda backend, since Lambda cannot be called directly from the public internet. The PDF also confirms that this lab uses **HTTP API** and that the next lab will compare it with **REST API**.

Understanding API Gateway in This Lab

API Gateway exists so a frontend application can send HTTP requests to a secure entry point instead of calling Lambda directly. In this lab, it acts as the middle layer that receives POST, GET, and DELETE requests and forwards each request to the correct Lambda function. This structure keeps the application secure, organized, and easy to expand later. Since the API is built as an **HTTP API**, the setup is lighter and faster for Lambda integration than a full **REST API** setup.

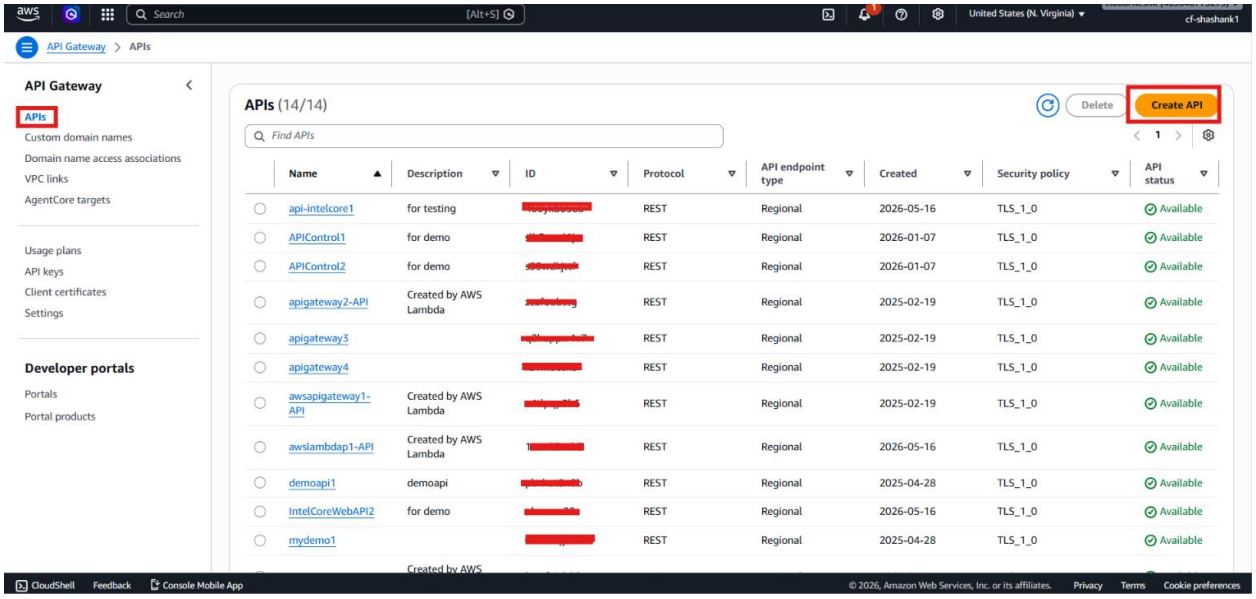
Example:

A mobile app wants to create a user named Kabir, read user 101, and delete user 101. Instead of talking directly to Lambda, the app sends POST /user, GET /user/101, and DELETE /user/101 to **API Gateway**. API Gateway then forwards each request to the matching Lambda function, which performs the DynamoDB operation.

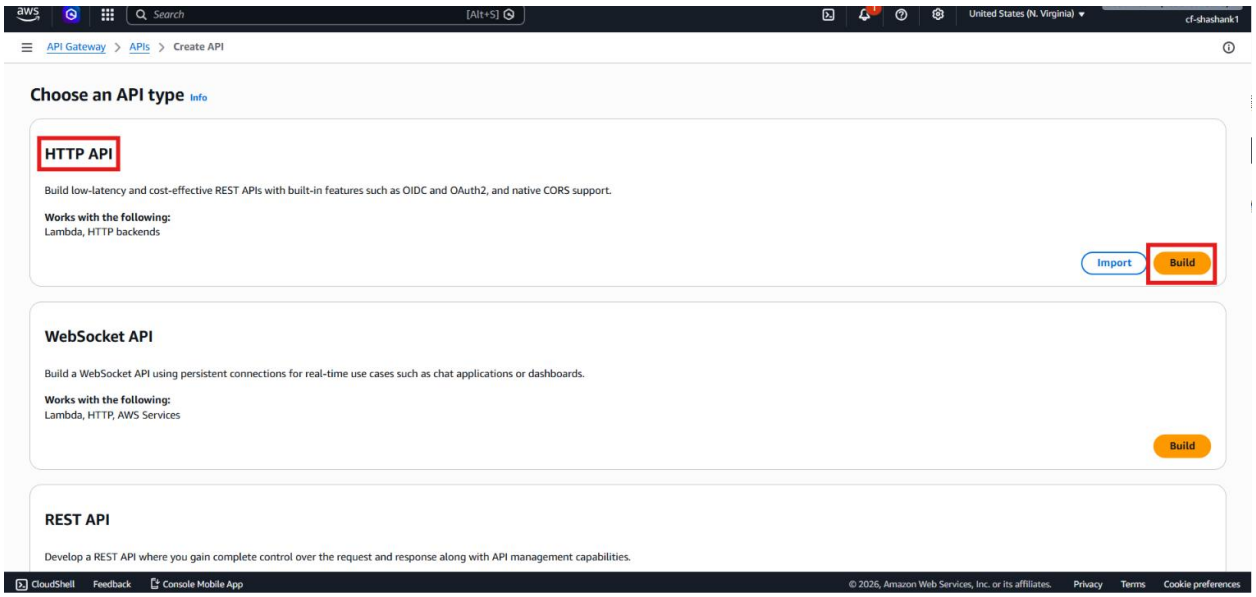
Lab Steps

Step 1 – Open API Gateway

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Click **Create API**.

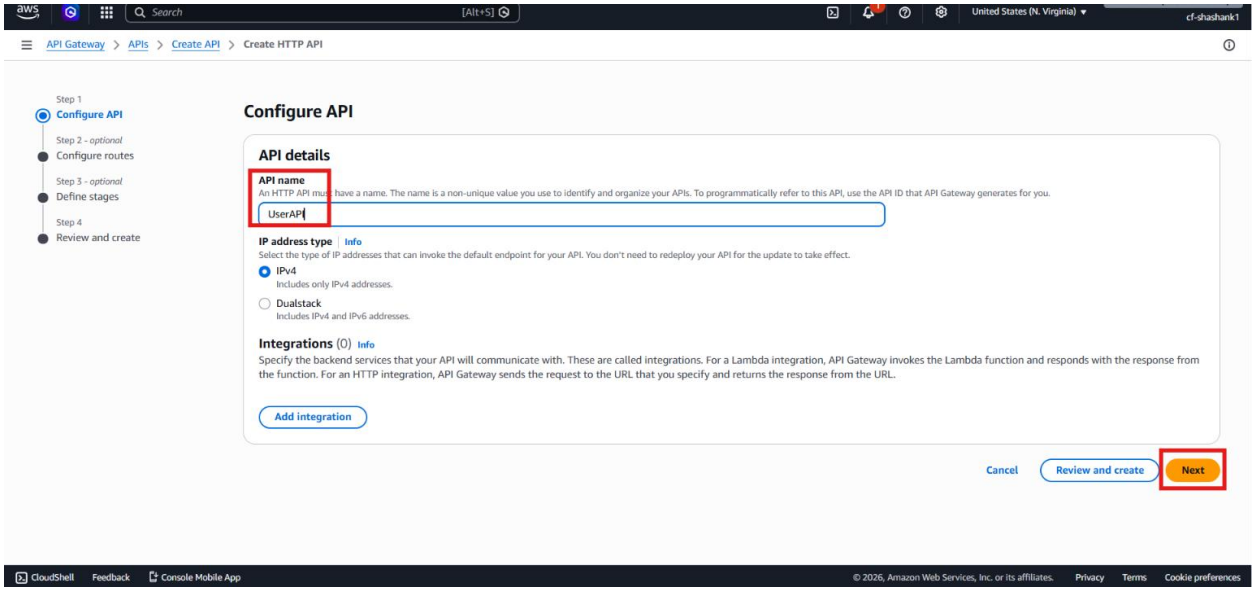


- Choose **HTTP API**.
- Click **Build**.

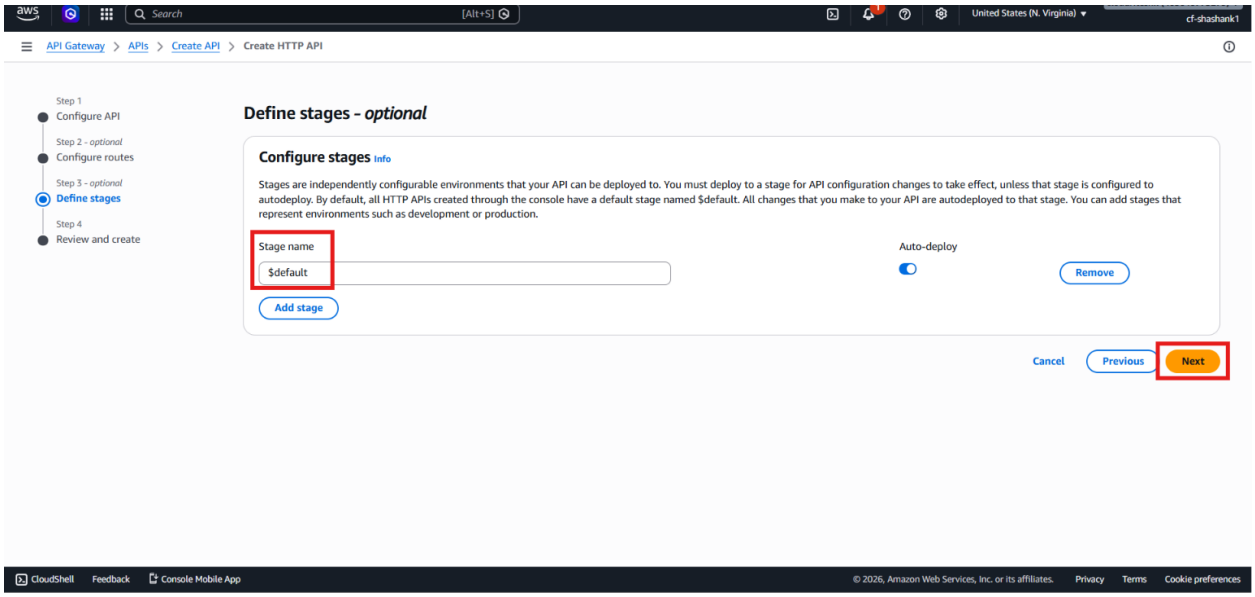


Step 2 – Create the HTTP API

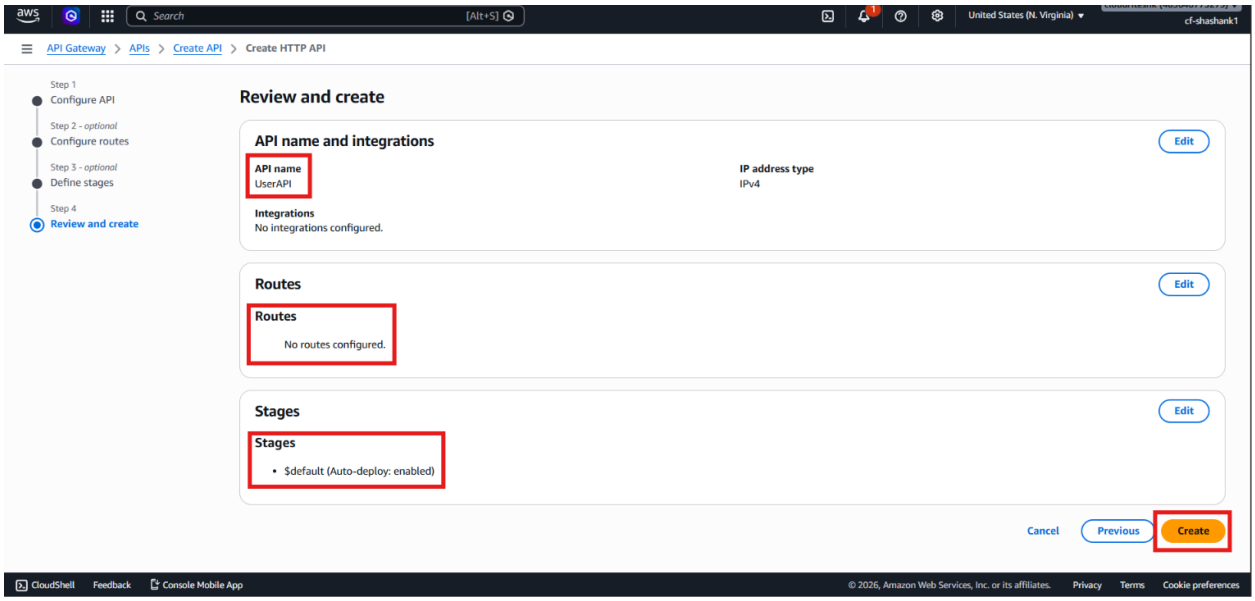
- Enter the API name: UserAPI
- Keep the default settings.



- Leave routes and stages unchanged for now.
- Click **Next**.

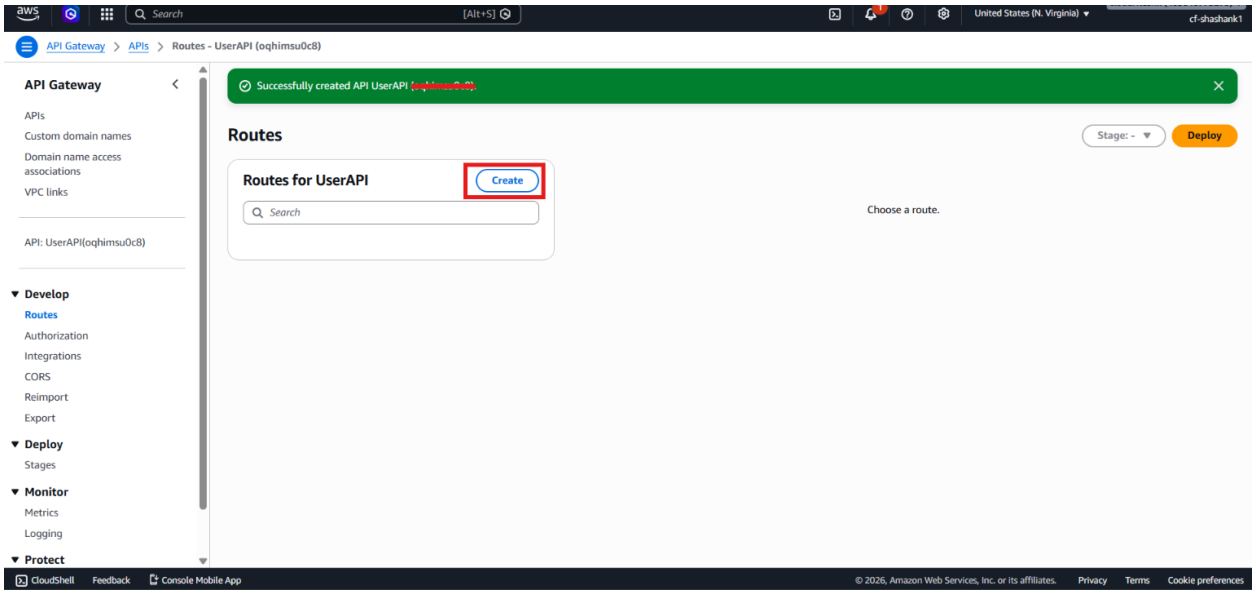


- Click **Create**.

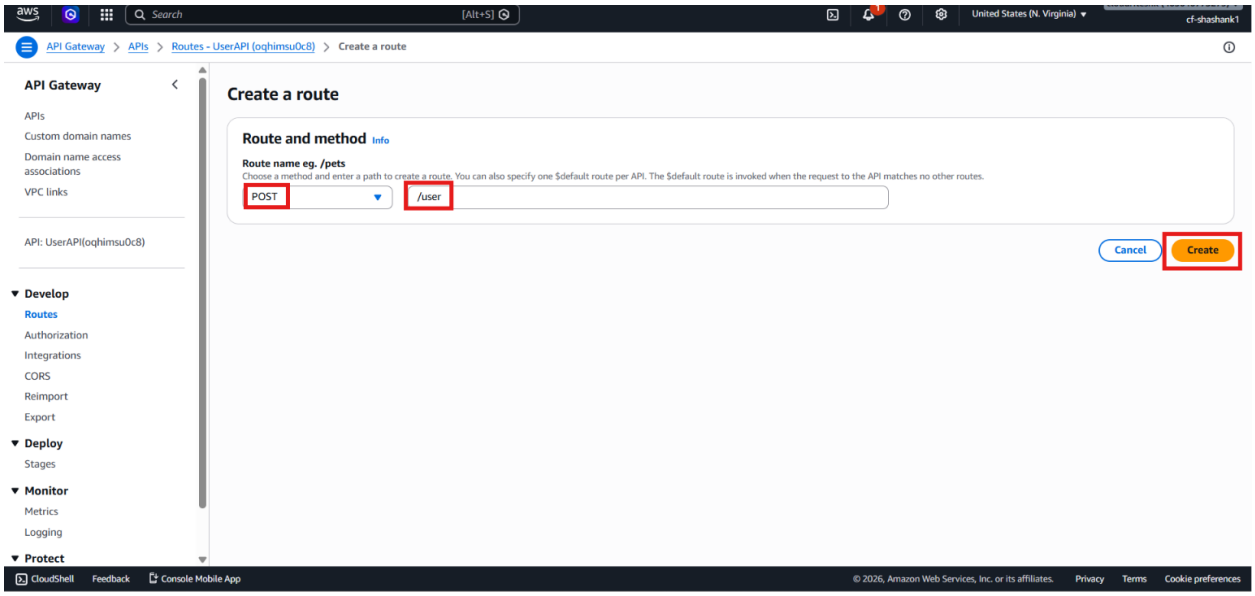


Step 3 – Create the Create User Route

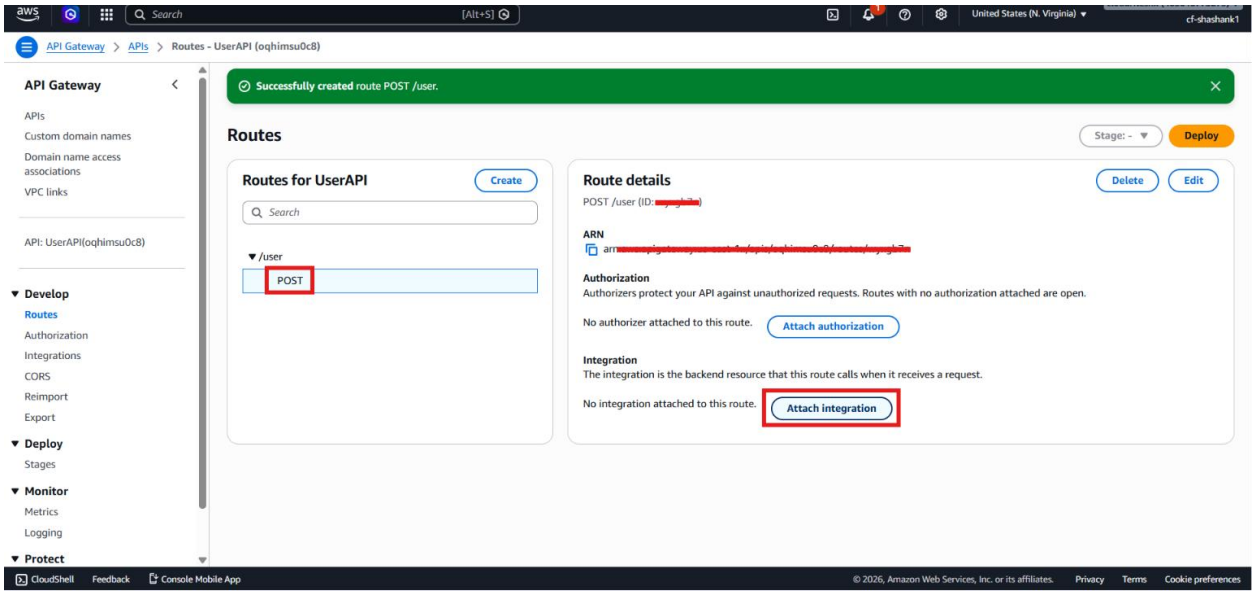
- Open **UserAPI**.
- Go to **Routes**.
- Click **Create**.



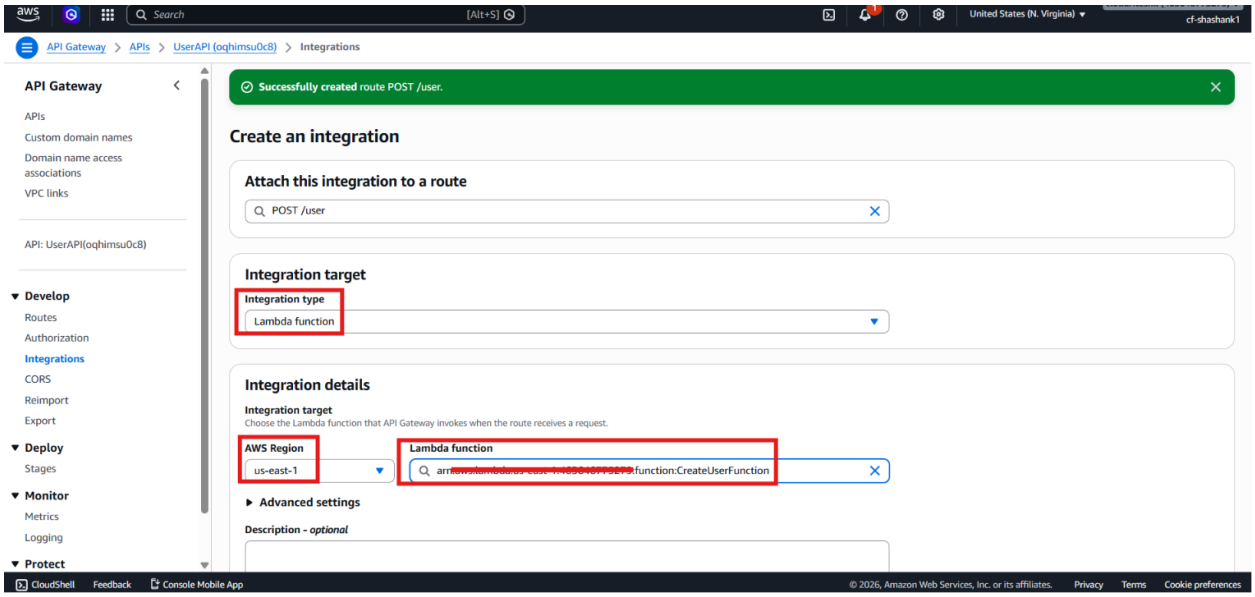
- Select **POST** as the method.
- Enter the route path: /user
- Click **Create**.



- Click **Attach integration**.



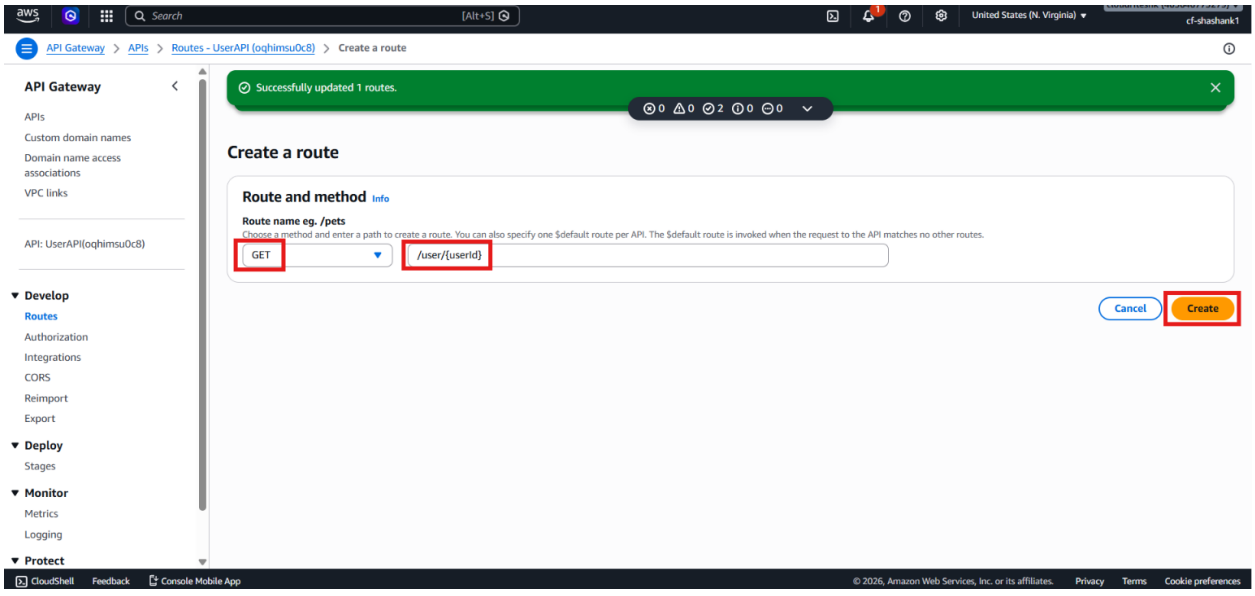
- Choose **Lambda function**.
- Select **createUserFunction**.



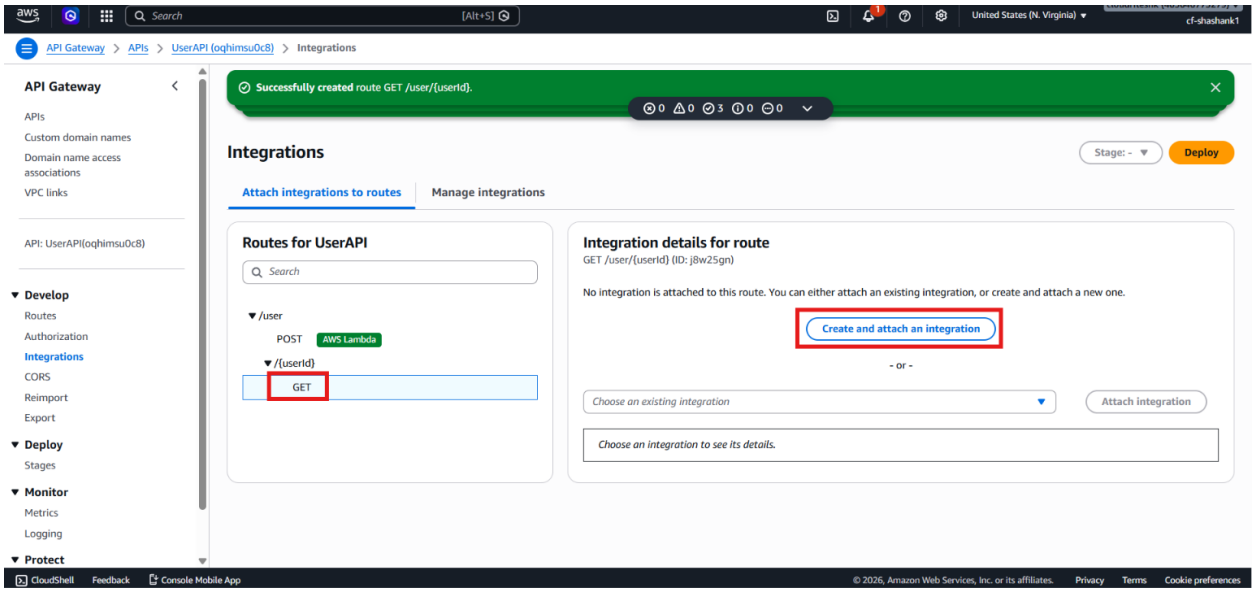
- Click **Create**.

Step 4 – Create the Get User Route

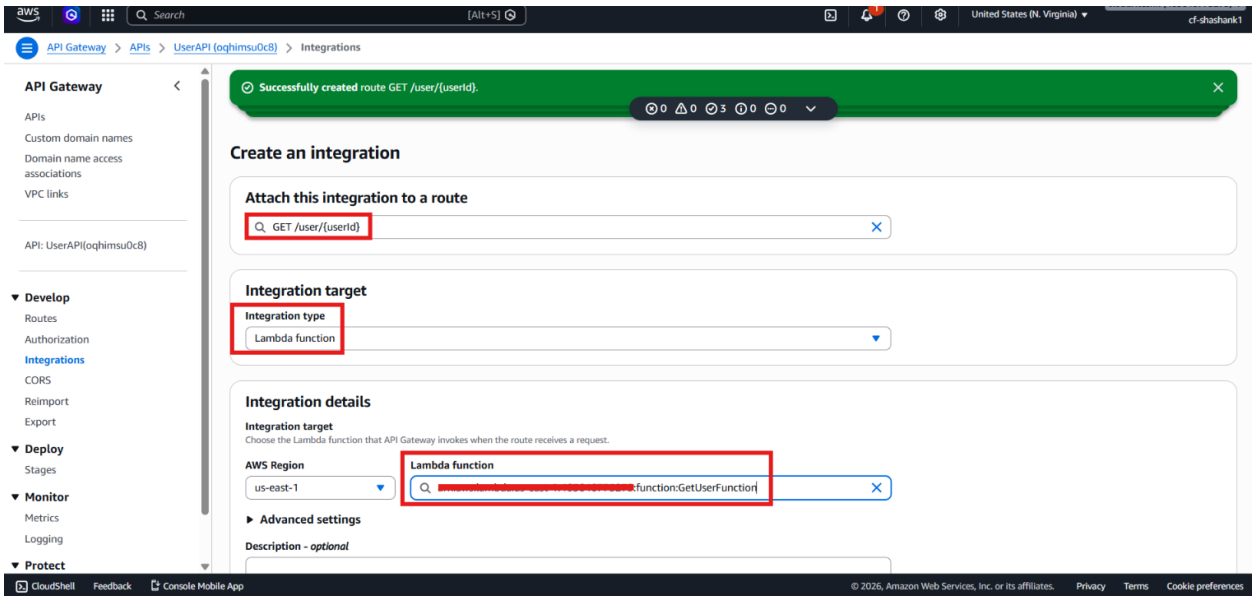
- Go back to **Routes**.
- Click **Create**.
- Select **GET** as the method.
- Enter the route path: /user/{userId}



- Click **Create**.
- Click **Attach integration**.



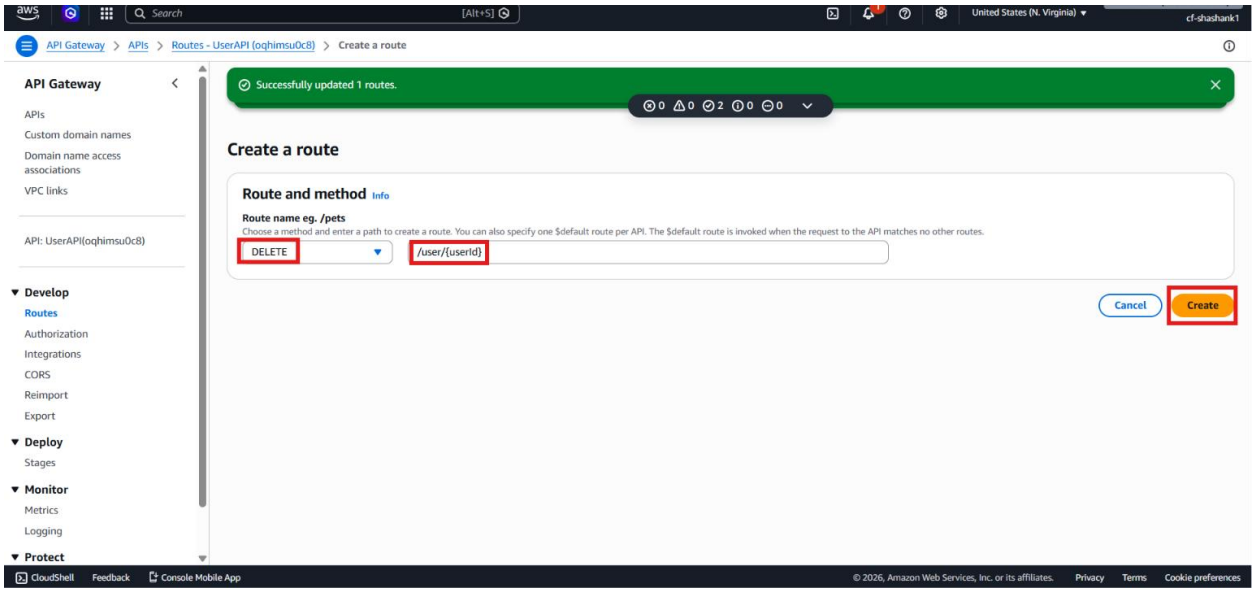
- Choose **Lambda function**.
- Select **getUserFunction**.



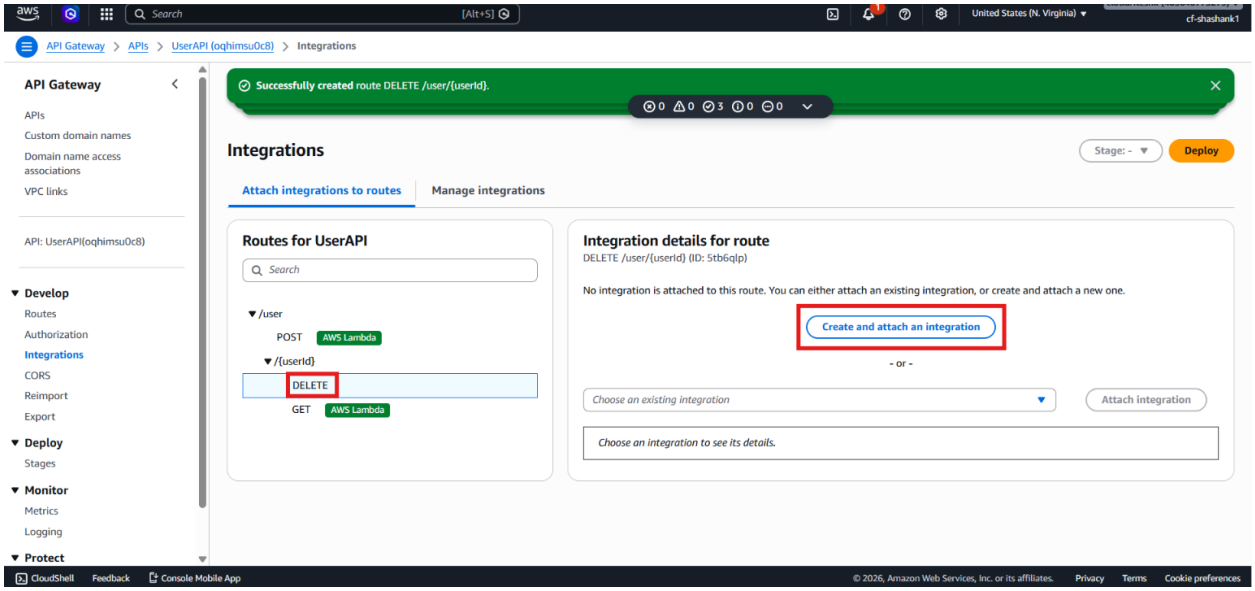
- Click **Create**.

Step 5 – Create the Delete User Route

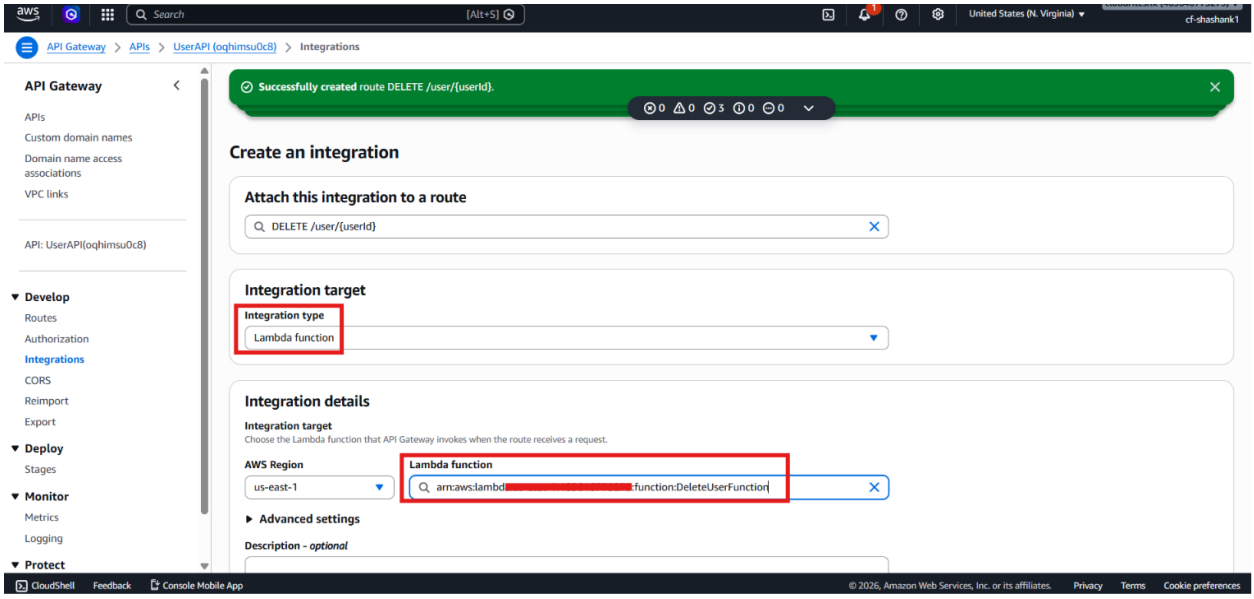
- Go back to **Routes**.
- Click **Create**.
- Select **DELETE** as the method.
- Enter the route path: /user/{userId}



- Click **Create**.
- Click **Attach integration**.



- Choose **Lambda function**.
- Select **deleteUserFunction**.



- Click **Create**.

Step 6 – Review the API Mapping

- Confirm the route mappings:
  - POST /user → **createUserFunction**
  - GET /user/{userId} → **getUserFunction**
  - DELETE /user/{userId} → **deleteUserFunction**
- Review the API trigger URL.
- Note that this is the URL the frontend or Postman will use to invoke the API.

Step 7 – Prepare for Testing

- Open the **Stages** tab.
- Copy the invoke URL.
- Keep the API ready for the final testing step.
- Prepare to use a frontend app or Postman in the next lab part.

Verification

- The **HTTP API** was created successfully in **API Gateway**.
- The route structure for create, read, and delete was defined.
- Each route was connected to the correct **AWS Lambda** function.
- The API is now acting as the bridge between the frontend and Lambda backend.
- The lab is ready for end-to-end testing in the next part